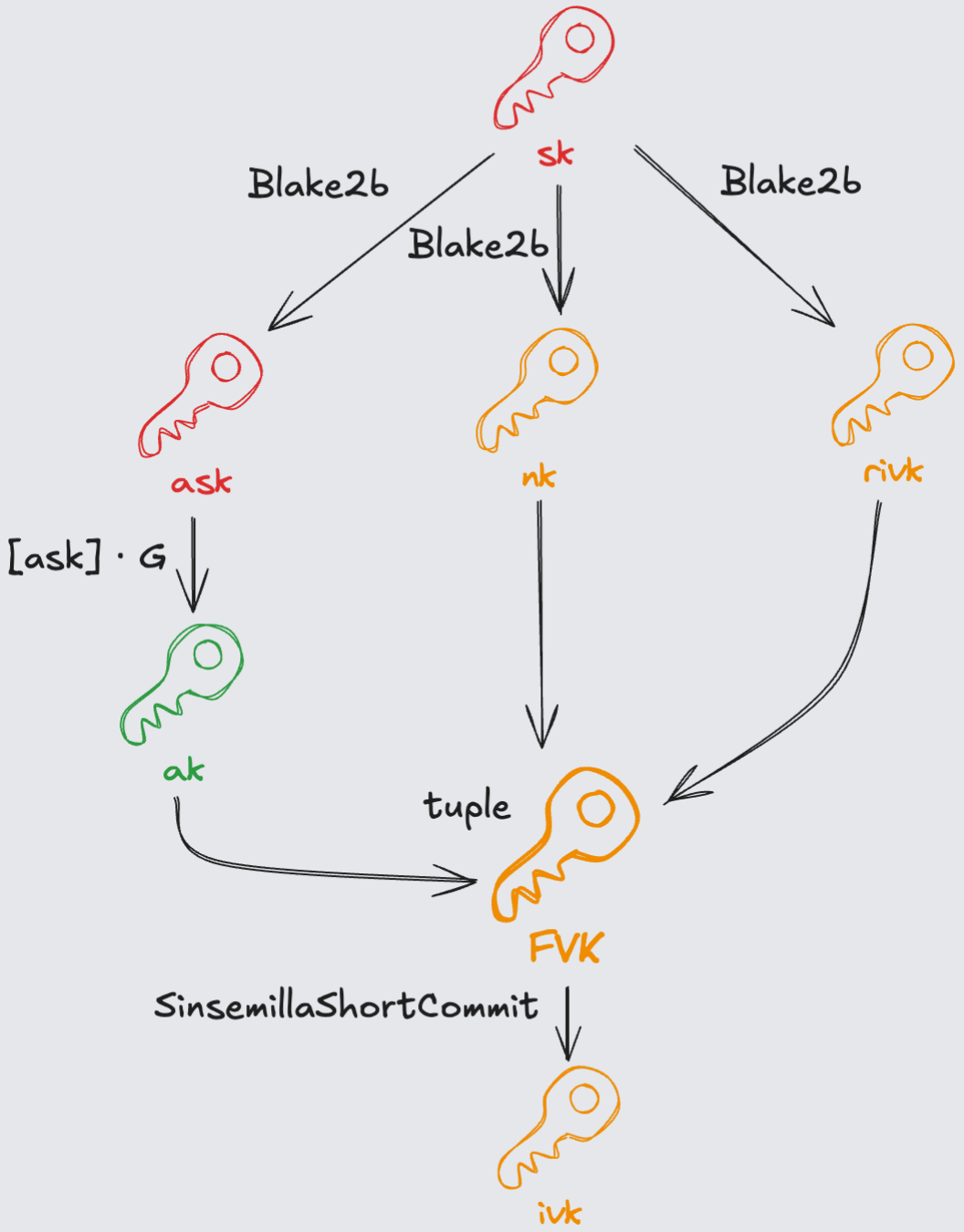
ZCash Orchard Circuits

Keys recap

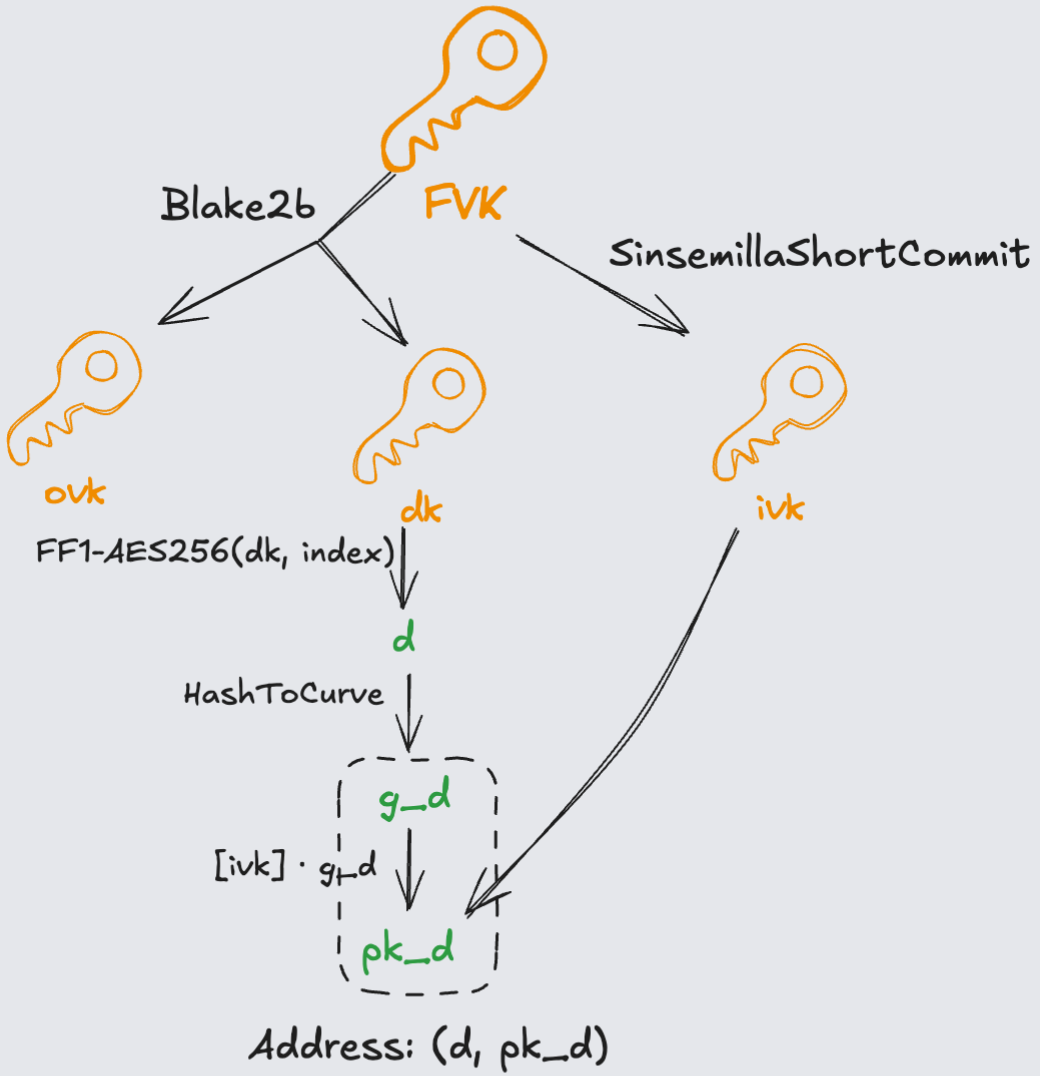
Account keys

- sk (**s**pending **k**ey)
- ask (**a**uthorizing **s**pending **k**ey)
- nk (**n**ullifier **k**ey)
- fvk (**f**ull **v**iewing **k**ey)
- rivk (**r**andomness for **i**ncoming **v**iewing **k**ey)
- ivk (**i**ncoming **v**iewing **k**ey)



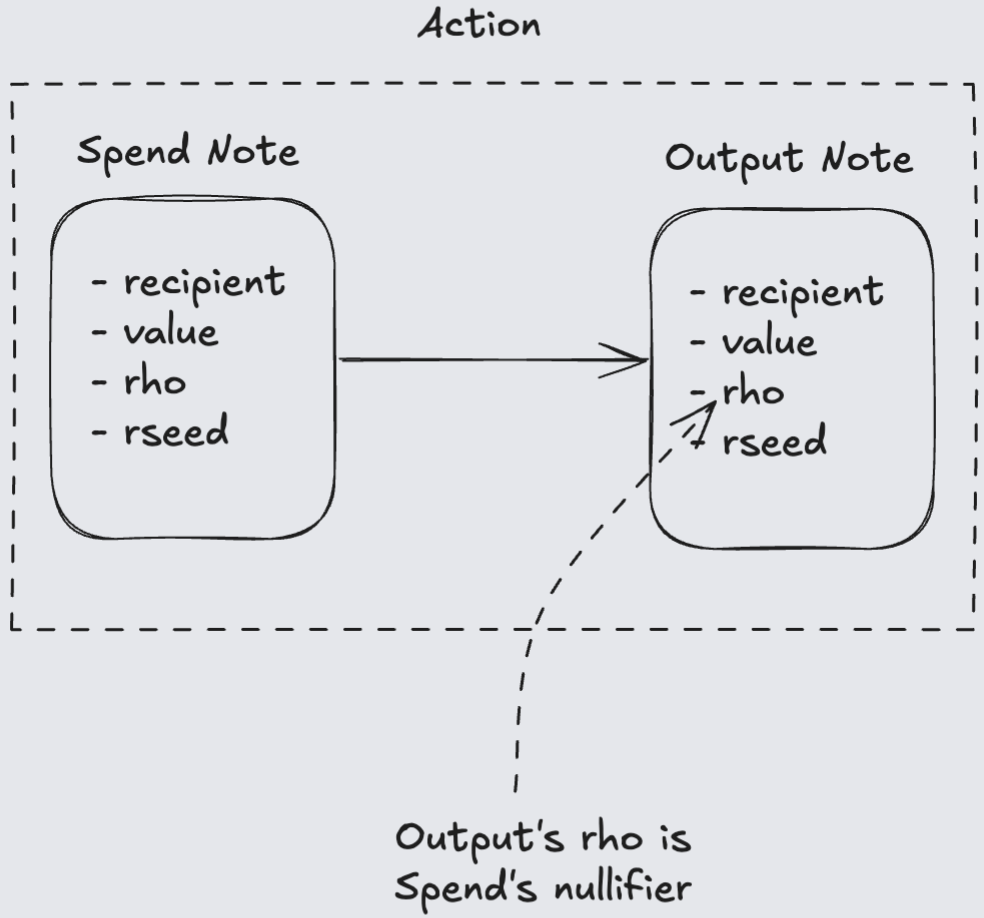
Account addresses

- dk (**d**iversifier **k**ey)
- ovk (**o**utgoing **v**iewing **k**ey)
- d (**d**iversifier)
- g_d (**g**enerator from **d**iversifier)
- pk_d (**p**ublic **k**ey, **d**iversified)



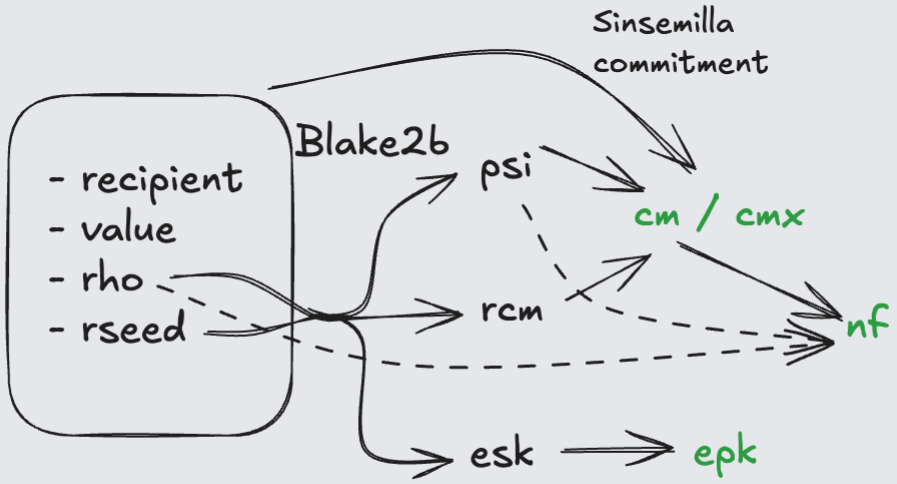
Note struct

- receiver – address of the note's owner
- value – the amount of ZEC (in zatoshis) this note is worth
- rho – prevents duplicating notes (as address, value, rseed could be duplicated)
- rseed – 32 random bytes

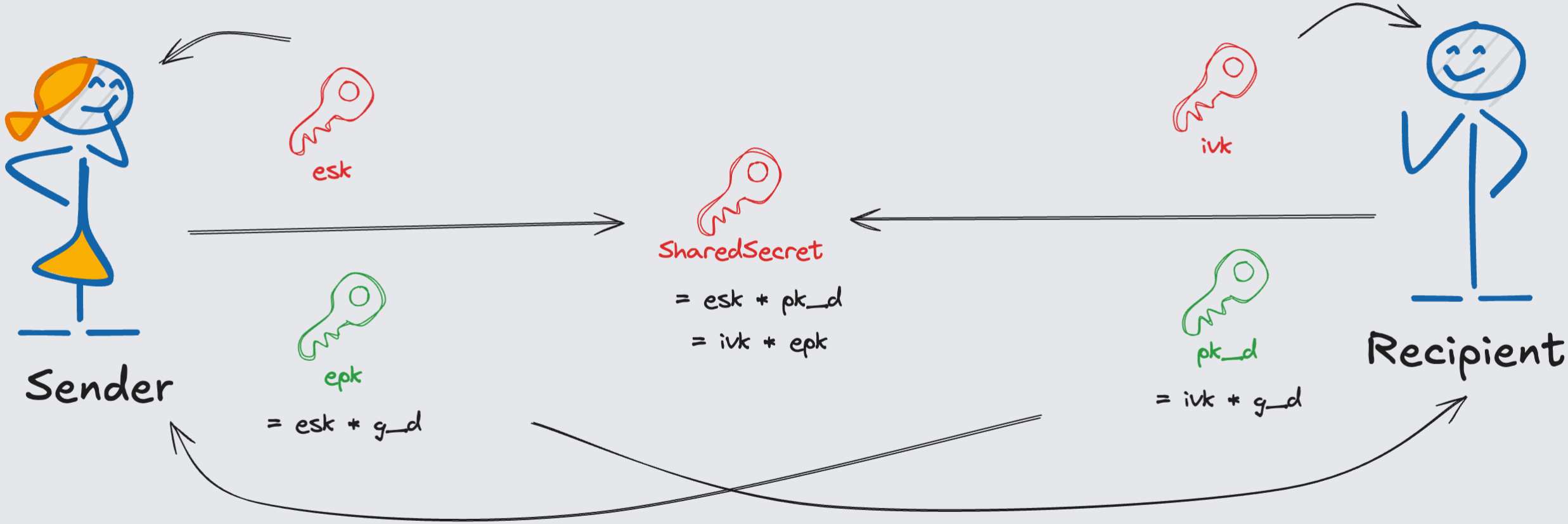


Note struct

- psi – random blinding factor
- rcm (**r**andomness for **c**ommit**m**ent)
- esk (**e**phemeral **s**ecret **k**ey)
- cm (note **c**ommit**m**ent)
- cmx (x coordinate of cm)
- nf (**n**ulli**f**ier)



ECDH


esk
ivk
SharedSecret
= esk * pk_d
= ivk * epk
epk
= esk * g_d
pk_d
= ivk * g_d
Sender
Recipient

Circuit

What does the circuit prove?

- Old note exists in the commitment tree: Merkle path verification
- Value commitment is correct: `cv = [v_old - v_new]V + [rcv]R`
- Nullifier derived correctly: `nf = Extract(cm + [Poseidon(nk,rho)+psi]K)`
- Spender knows the spend key: `rk = [alpha]G + ak`
- Address matches the viewing key: `pk_d = [CommitIvk(ak,nk,rivk)] g_d`
- Old note commitment is correct: recompute `cm_old` from note fields
- New note commitment is correct: recompute `cm_new`, expose `cmx`
- Spend/output flags are consistent: `v_old != 0 => enable_spend = 1`

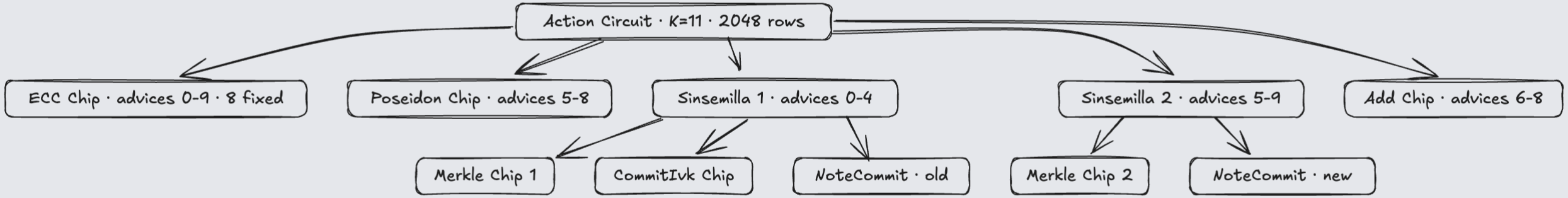
Public inputs

Index	Name	What it is
0	anchor	Merkle root of the commitment tree
1-2	cv_net x,y	Value commitment (hides the transferred amount)
3	nf_old	Nullifier of the spent note
4-5	rk x,y	Randomized spend verification key
6	cmx	New note commitment (x-coordinate)
7	enable_spend	Flag: are spends allowed in this bundle?
8	enable_output	Flag: are outputs allowed?

Private witness

- Old note: `g_d, pk_d, v, rho, psi, rcm, cm`
- Merkle: `path[32], pos`
- Keys: `ak, nk, rivk, alpha`
- New note: `g_d_new, pk_d_new, v_new, psi_new, rcm_new`
- Value: `rcv`

Chip Architecture



Column sharing layout

Column	0	1	2	3	4	5	6	7	8	9
ECC	x_a	x_p	bits	lam1	lam2	x_a	x_p	bits	lam1	lam2
Sinsemilla 1	x_a	x_p	bits	lam1	lam2	wit	-	-	-	-
Sinsemilla 2	-	-	-	-	-	x_a	x_p	bits	lam1	lam2
Poseidon	-	-	-	-	-	sbox	st0	st1	st2	-
AddChip	-	-	-	-	-	-	c	a	b	-
RangeCheck	-	-	-	-	-	-	-	-	-	z

Synthesize flow

1. Witness private inputs
2. Merkle path → `root`
3. Value commitment → public `cv_net`
4. Nullifier → public `nf_old`
5. Spend authority → public `rk`
6. Address integrity → internal `pk_d_old` check
7. Old note commitment → internal `cm_old` check
8. New note commitment → public `cmx`
9. Orchard gate → ties it all together

Sinsemilla

Sinsemilla is an algebraic hash optimized for Halo2 circuits. It processes messages as 10-bit words using a lookup table of 1024 pre-computed curve points.

Algorithm

for message `m = m_1 || m_2 || ... || m_n` (each `m_i` is 10 bits):

```
Q_0 = domain separator point
for i in 1..=n:
    Q_i = (Q_{i-1} + S(m_i)) + Q_{i-1}
```

`S(m_i)` is looked up from the generator table. Each step is one row in the circuit.

Sinsemilla problem: field elements vs bits

Both CommitIvk and NoteCommit face the same fundamental tension:

- Sinsemilla sees 10-bit words. It doesn't know they represent field elements.
- The protocol requires hashing specific field elements (`ak` , `nk` , `x(g_d)` , `rho` , `psi`).
- The circuit must prove the bits fed to Sinsemilla actually correspond to the claimed field elements.

Running sum decomposes each piece into 10-bit words:

```
z_0 = piece_value,   z_{i+1} = (z_i - word_i) / 2^10,   z_n = 0
```

If `z_n = 0` , the piece was fully consumed -- it fits in `n * 10` bits.

Sinsemilla problem: field elements vs bits

This creates two problems:

1. Decomposition: prove the Sinsemilla message pieces are a valid bit-decomposition of the input.
2. Canonicity: prove the bit representation is the unique canonical one (not some mod-p equivalent with different bits).

Every constraint in CommitIvk and NoteCommit exists for one of these two reasons.

The canonicity technique

The Pallas base field has modulus: `p = 2^254 + t_p`

Goal: Prove `x < t_p` (where `t_p ≈ 2^125`).

Technique: Prove that `x + 2^K − t_p` fits in `K` bits.

Case	x + 2^K - t_p	Fits in K bits?
`x < t_p` (valid)	`< 2^K`	Yes, running sum = 0
`x >= t_p` (invalid)	`>= 2^K`	No, running sum ≠ 0